

How Can State Space Models Enhance Machine Learning on Graphs?

Yinan Huang[✉], Siqi Miao, and Pan Li[✉]

Abstract—Message Passing Neural Networks are known to struggle with limited expressivity and capturing long-range dependencies. While Graph Transformers alleviate these issues with global attention modules, their quadratic complexity limits efficiency. Recently, State Space Models have emerged as a compelling alternative to attention for sequence modeling to 1) capture long-range dependencies, 2) enable computational efficiency, and 3) generalize well across varying sequence lengths. However, extending SSMs to graphs presents unique challenges due to the lack of canonical node ordering. This work studies how to leverage the insights of SSM design to improve learning on graph-structured data and proposes Graph State Space Convolution (GSSC). Using global permutation-equivariant set aggregation and factorizable graph kernels based on relative node distances, GSSC retains the three advantages of SSMs. Across 11 real-world, widely used graph benchmark datasets, GSSC achieves the best results on 5 datasets with significant improvements over the state-of-the-art baselines and competitive performance on the other 6. Our findings highlight the potential of building powerful and scalable models upon GSSC for graph machine learning.

Index Terms—Graph neural networks, state space models.

I. INTRODUCTION

Machine learning for graph-structured data has numerous applications in molecular graphs [1], [2], drug discovery [3], [4], and social networks [5], [6]. In recent years, Message Passing Neural Networks (MPNNs) have been arguably the most popular neural architecture for graphs [7], [8], but they also suffer from many limitations, including restricted expressive power [9], [10], over-squashing [11], and over-smoothing [12]. These limitations could harm the models' performance. For example, MPNNs cannot capture long-range dependencies [13] or detect subgraphs like cycles that are important in forming ring systems of molecular graphs [14].

Graph transformers, adapted from sequence modeling transformers [15], have gained increasing interest for potentially addressing MPNN's limitations. By attending to all nodes, graph transformers are inherently able to capture long-range dependencies. However, the global attention mechanism ignores graph structures and thus requires incorporating positional encodings (PEs) of nodes that encode graph structural information [16]. For example, the information of relative distance between nodes has been leveraged in attention computation [17], [18], [19]. Moreover, the full attention computation scales quadratically in terms of the length of the sequence or the number of nodes in the graph. This computational challenge motivates the study of linear-time

transformers by incorporating techniques such as low-rank [20], [21], sparse approximations [22], [23], or Taylor expansion [24] of the attention matrix. Some specific adaptations of scalable transformers to large-scale graphs are also proposed [25], [26], [27], [28], [29]. Nevertheless, none of these variants have been proven to be consistently effective across different domains [30].

State Space Models (SSMs) [31], [32], [33] have recently demonstrated promising potentials for sequence modeling. SSMs can be seen as a hybrid of RNNs and CNNs. It is a temporal convolution that preserves translation invariance and allows good generalization to sequences longer than those used for training. Meanwhile, it allows capturing *long-range dependencies* both theoretically and empirically [32], [34], [35]. Finally, it can be efficiently computed in *linear or near-linear time* via the recurrence mode or the parallelizable operations.

Given the great potential of SSMs, there is increasing interest in using them to enhance learning on graphs [36], [37]. The main technical challenge is that SSMs are defined on sequences that are ordered and causal. Yet, graphs have complex topology, and no canonical node ordering can be found. Naive tokenization (e.g., sorting nodes into a sequence in some ways) breaks the inductive bias - permutation symmetry - of graphs, which consequently cannot faithfully represent graph topology, and may suffer from poor generalization.

Instead of simply tokenizing graphs and directly applying existing SSMs for sequences (which may break the symmetry), we argue that SSMs for graphs should inherit their advantages in capturing long-range dependencies and being efficient, and meanwhile they should preserve the permutation symmetry of graphs to achieve good generalization.

In this work, we identify that the key components that enable SSMs for sequences to be long-range, efficient, and well-generalized to longer sequences, is the use of a global, factorizable, and translation-invariant kernel that depends on relative distances between tokens. This relative-distance kernel can be factorized into the product of absolute positions, crucial to achieving linear-time complexity.

This observation motivates us to design Graph State Space Convolution (GSSC) that enhance canonical learning on graph models in the following way: (1) it leverages a global *permutation equivariant set aggregation* that incorporates all nodes in the graph; (2) the aggregation weights of set elements rely on relative distances between nodes, which can be factorized into the “absolute positions” of the corresponding nodes. By design, the resulting GSSC is inherently permutation equivariant, long-range, and linear-time. Besides, GSSC improves the expressive power of MPNNs in terms of graph distinguishability and substructure counting.

Empirically, our experiments demonstrate the expressivity of GSSC-enhanced GNNs via graph substructure counting and validate its capability of capturing long-range dependencies on Long Range Graph Benchmark [13]. Results on 11 widely used graph learning benchmarks [13], [38], [39] also show the consistently superior performance of GSSC, where GSSC achieves best results on 5 out of 11 datasets with all significant improvements compared to the state-of-the-art baselines and high competitive results on the other 6 datasets. Moreover, it offers better scalability than the standard graph transformers in terms of training and inference time.

Received 21 September 2025; revised 13 February 2026; accepted 28 March 2026. Date of publication 3 April 2026; date of current version 6 July 2026. The work was supported in part by NSF under Award PHY-2117997, Award IIS-2239565, Award IIS-2428777, and Award IIS-2435957, in part by JPMC faculty award 2023, and in part by NVIDIA Academic Grant Program. Recommended for acceptance by M.-L. Zhang. (Siqi Miao and Yinan Huang contributed equally to this work.) (Corresponding author: Pan Li.)

The authors are with the School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta GA 30332 USA (e-mail: yhuang903@gatech.edu; siqi.miao@gatech.edu; panli@gatech.edu).

Our code is available at <https://github.com/Graph-COM/GSSC>.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TPAMI.2026.3680437>, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2026.3680437

II. PRELIMINARIES

Graphs: $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is an undirected graph, where $\mathcal{V}$ is the n -sized node set and $\mathcal{E}$ is the edge set. Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be the adjacency matrix and $\mathbf{D} = \text{diag}([\sum_j \mathbf{A}_{1,j}, \dots, \sum_j \mathbf{A}_{n,j}])$ be the diagonal degree matrix. The (normalized) graph Laplacian is defined by $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$ where $\mathbf{I}$ is the n by n identity matrix.

State Space Models: SSM is a continuous system that maps an input function $x(t)$ to output $h(t)$ via the first-order differential equation: $\frac{d}{dt} h(t) = \mathbf{A}h(t) + \mathbf{B}x(t)$. This system can be discretized (e.g., via zero-order hold [33]) with time step Δ . Suppose $h_i := h(i \cdot \Delta)$ and $x_i := x(i \cdot \Delta)$. The discrete SSM becomes a recurrence process:

$$h_i = \bar{\mathbf{A}}h_{i-1} + \bar{\mathbf{B}}x_i, \quad (1)$$

where $\bar{\mathbf{A}} = f_{\mathbf{A}}(\Delta, \mathbf{A})$ and $\bar{\mathbf{B}} = f_{\mathbf{B}}(\Delta, \mathbf{B})$ depend on the specific discretization rule. The recurrence (1) can be computed equivalently by a global convolution:

$$h_i = \sum_{j=1}^i \bar{\mathbf{A}}^{i-j} \bar{\mathbf{B}}x_j. \quad (2)$$

In the remaining of this paper, we call convolution (2) **state space convolution (SSC)**. Notably, the state space model enjoys three key advantages simultaneously:

- **Translation equivariance:** Input translation $x_i \rightarrow x_{i+\delta}$ yields the same output translation $h_i \rightarrow h_{i+\delta}$.
- **Long-range dependencies:** The feature h_i of i -th token depends on all preceding tokens' features $x_1, x_2, \dots, x_{i-1}$. With a properly chosen structured matrix $\bar{\mathbf{A}}$ [32], [34], the gradient norm $\|\partial h_i / \partial x_{i-j}\|$ does not decay as j goes large. This is different to the fixed-size receptive field in CNNs and the vanishing gradient norm in RNNs [40].
- **Computational efficiency and parallelism:** To computing $h_1, \dots, h_n$, it adopts either recurrence (1) or convolution (2). Near-linear-time algorithms are introduced to provide parallelism for efficient training, e.g., FFT [32], [41], block-decomposition matrix multiplication [42] for convolution, and parallel scan [33] for recurrence.

Notation: Let $\langle x, y \rangle = \sum_i x_i y_i$ be the inner product, and $x \odot y = (x_1 y_1, x_2 y_2, \dots)$ be the element-wise product. We denote the feature hidden dimension by m , and the dimension of positional encodings by d .

III. GRAPH STATE SPACE CONVOLUTION

A. Generalizing State Space Convolution to Graphs

Like standard SSC (Eq. (2)), the desired graph SSC should keep the good capturing of long-range dependencies as well as linear-time complexity and parallelizability. Meanwhile, permutation equivariance as a strong inductive bias of graph-structured data should be preserved by the model as well to improve generalization.

Our key observations of SSC start with the fact that the convolution kernel $\bar{\mathbf{A}}^{i-j}$ encoding the relative distance $i - j$ between token i and j allows for a natural factorization:

$$h_i = \sum_{j=1}^i \bar{\mathbf{A}}^{i-j} \bar{\mathbf{B}}x_j = \bar{\mathbf{A}}^i \sum_{j=1}^i \bar{\mathbf{A}}^{-j} \bar{\mathbf{B}}x_j. \quad (3)$$

Generally, $\sum_j K(i, j)x_j$ with a generic kernel $K(i, j)$ requires quadratic-time computation and may not capture translation invariant patterns for variable-length generalization. For SSC (Eq. (3)), however,

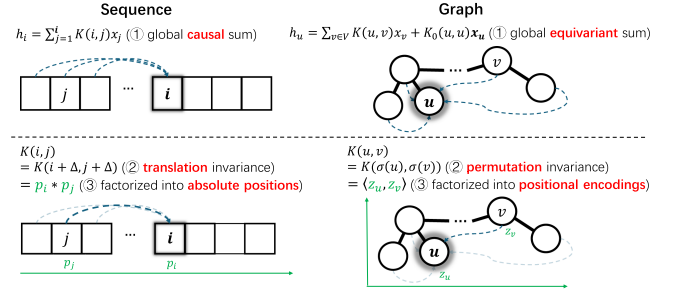


Fig. 1. Comparison of Sequence State Space Convolution (left) and Graph State Space Convolution (right).

it captures translation invariant patterns by adopting a **translation-invariant** kernel $K(i - j)$ that only depends on the relative distance $i - j$, enabling generalization power to sequences even longer than those for training [43]. Moreover, SSC attains computational efficiency by leveraging the **factorability** of its design of the relative distance kernel $K(i - j) = \bar{\mathbf{A}}^{i-j} = \bar{\mathbf{A}}^i \cdot \bar{\mathbf{A}}^{-j}$, where the factors only depend on the *absolute positions* of token i and j . To compute h , one constructs $[\sum_{j=1}^1 \bar{\mathbf{A}}^{-j} \bar{\mathbf{B}}x_j, \dots, \sum_{j=1}^n \bar{\mathbf{A}}^{-j} \bar{\mathbf{B}}x_j]$ using prefix sum with complexity $O(n)$, and then readout h_i by multiplying $\bar{\mathbf{A}}^i$ for all $i = 1, 2, \dots, n$ in parallel with complexity $O(n)$. Finally, the **global causal sum** $\sum_{j \leq i}$ helps capture global dependencies. In general, the advantages of SSC are attributed to these three aspects.

Inspired by these insights, a natural generalization of SSC to graphs, written generally as $h_u = \sum_{v \in \mathcal{V}} K(u, v)x_v$, is expected to adopt a **permutation-invariant** kernel $K(u, v)$, $u, v \in \mathcal{V}$ to capture the inductive bias of graphs. The kernel should be **factorizable** $K(u, v) = z_u^\top z_v$ with certain notion of node absolute position for computational efficiency, and the convolution should perform **global pooling** across the entire graph to capture global dependencies. Note that causal sum $\sum_{j \leq i}$ is replaced by global pooling $\sum_{v \in \mathcal{V}}$ due to the lack of node orderings. See Fig. 1 for an illustration.

Fortunately, a systematic strategy can be adopted to design such kernels. First, the kernel $K(v, u)$ should depend on some notions of relative distance between nodes to capture graph topology and preserve permutation invariance. The choices include but are not limited to shortest-path distance, random walk landing probability [44], heat (diffusion) distance [45], resistance distance [46], etc. Many of them have been widely adopted as edge features for existing models, e.g., GNNs [17], [47], [48], [49] and graph transformers [16], [50], [51]. More importantly, all these kernels can be factorized into some weighted inner product of Laplacian eigenvectors [52]. Here, Laplacian eigenvectors, also known as Laplacian PE (LPE) [18], [53], play the role of the absolute positions of nodes in the graph. Formally, consider the eigendecomposition $\mathbf{L} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top$ and let $p_u = [\mathbf{V}_{u,:}]^\top$ be the LPE for node u . Then a relative-distance kernel $K(u, v)$ can be generally factorized into $K(u, v) = p_u^\top (\phi(\mathbf{\Lambda}) \odot p_v)$ for certain functions ϕ . For instance, diffusion kernel satisfies $[\phi(\mathbf{\Lambda})]_k = \exp(-t\lambda_k)$ for some time parameter t .

Graph State Space Convolution (GSSC): With the observations above, we are ready to present GSSC. Given the input node features $x_u \in \mathbb{R}^m$, the d -dim Laplacian positional encodings $p_u = [\mathbf{V}_{u,1:d}]^\top \in \mathbb{R}^d$, and the corresponding d eigenvalues $\mathbf{\Lambda}_d = [\lambda_1, \dots, \lambda_d]^\top$, the output node representations $h_u \in \mathbb{R}^m$ follow

$$h_u = \sum_{v \in \mathcal{V}} \langle z_u \mathbf{W}_q, z_v \mathbf{W}_k \rangle \odot \mathbf{W}_o x_v + \langle z_u \mathbf{W}_{sq}, z_u \mathbf{W}_{sk} \rangle \odot \mathbf{W}_s x_u, \quad (4)$$

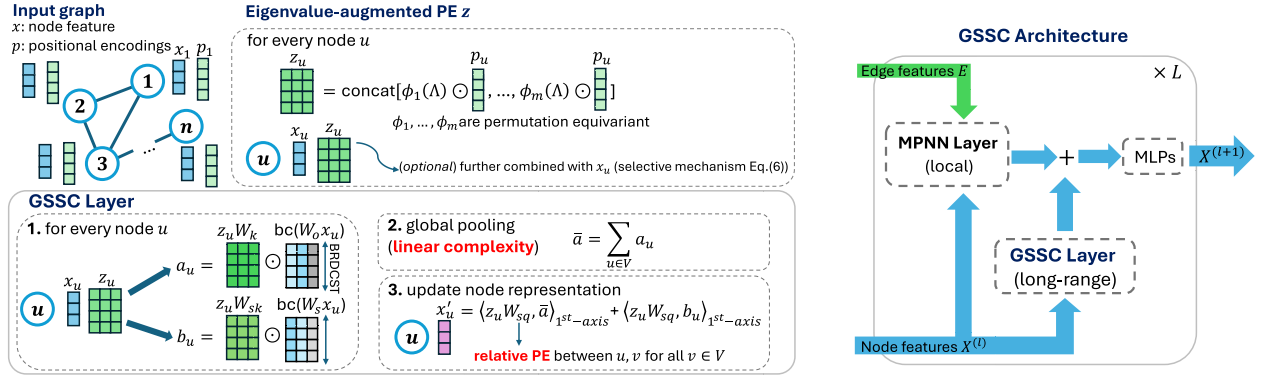


Fig. 2. Illustration of Graph State Space Convolution (GSSC) layer (left) and final architecture (right). The broadcasting operator bc broadcasts vector $x \in \mathbb{R}^m$ to $\mathbb{R}^{d \times m}$ in the first dimension.

$$= \left\langle z_u \mathbf{W}_q, \sum_{v \in \mathcal{V}} z_v \mathbf{W}_k \odot \mathbf{W}_o x_v \right\rangle + \langle z_u \mathbf{W}_{sq}, z_u \mathbf{W}_{sk} \rangle \odot \mathbf{W}_s x_u. \quad (5)$$

Here $z_u = [\phi_1(\Lambda_d) \odot p_u, \dots, \phi_m(\Lambda_d) \odot p_u] \in \mathbb{R}^{d \times m}$ represents the eigenvalue-augmented PEs from raw d -dim PEs p_u , and $\phi_\ell : \mathbb{R}^d \rightarrow \mathbb{R}^d$ are learnable permutation equivariant functions w.r.t. d -dim axis (i.e., equivariant to permutation of eigenvalues, we use DeepSets [54] in our experiments). All $\mathbf{W} \in \mathbb{R}^{m \times m}$ with different subscripts are learnable weight matrices. The inner product $\langle z_u \mathbf{W}_q, z_v \mathbf{W}_k \rangle \in \mathbb{R}^m$ only sums over the first d -dim axis. The term $z_u \mathbf{W}_{sq} \odot \mathbf{W}_s x_u$ in Eq.(5) should be interpreted as first broadcasting $\mathbf{W}_s x_u$ from $\mathbb{R}^m$ to $\mathbb{R}^{d \times m}$ and then performing element-wise products. Note that GSSC is a generalization of SSC (Eq. (3)) in the sense that:

- *Absolute position*: absolute position $\bar{A}^i$ is replaced by positional encodings z_u ;
- *factorizable kernel*: the kernel $\bar{A}^i \bar{A}^{-j}$ is replaced by the inner product of graph positional encodings $\langle z_u \mathbf{W}_q, z_v \mathbf{W}_k \rangle$.
- *global sum*: causal sum $\sum_{j=1}^i \bar{A}^{-j} \bar{B} x_j$ is replaced by an equivariant global pooling. As graphs are not causal, a node u only distinguishes itself (node u) from other nodes.

The term $z_u \mathbf{W}_{sq} \odot \mathbf{W}_s x_u$ is for node u itself, and $(\sum_{v \in \mathcal{V}} z_v \mathbf{W}_k) \odot \mathbf{W}_o x_v$ is for all nodes in the graph.

Proposition III.1 suggests that GSSC with learnable ϕ can capture long-range dependencies.

Proposition III.1. There exists ϕ such that for GSSC Eq.(4), the gradient norm $\|\partial h_u / \partial x_v\|$ does not decay as $\text{spd}(u, v)$ grows, where spd denotes the shortest path distance.

Remark III.1. The design of GSSC yields four key properties: (1) permutation equivariance: permuting node indices reorders h_u correspondingly; (2) long-range dependencies: supported by Proposition III.1; (3) linear-time: computing $h_1, \dots, h_n$ from $x_1, \dots, x_n$ costs $\mathcal{O}(nmd)$, where n is the number of nodes and m, d are hidden and positional dimensions; (4) stability: perturbations to the graph Laplacian induce controlled changes in the output, since permutation equivariance and smoothness of ϕ_ℓ ensure stability of $\langle z_u \mathbf{W}_q, z_v \mathbf{W}_k \rangle$ and hence the model output [18], [55]. Stability extends permutation equivariance and is crucial for out-of-distribution generalization [55].

B. Hybrid Modeling: Enhance MPNNs With GSSC

SSMs, by definition, lack a natural mechanism for integrating token-pairwise features and thus cannot utilize edge features. Therefore,

SSMs are better used to enhance graph learning models that can take edge features than standalone ones. For example, existing graph Mamba adopt a local graph encoder such as MPNNs to incorporate edge features [36], [37]. We follow this tradition by adopting a hybrid architecture combining GSSC and MPNN layers. This hybrid offers several advantages in both theory and practice: (1) MPNNs encode local edge information, while GSSC layers capture long-range global interactions, forming a complementary relationship between the two; (2) the combination retains linear-time complexity, unlike the quadratic complexity incurred by combining full attention with MPNNs in graph transformers [16], [56]; (3) the expressivity of GSSC and MPNNs complements each other, yielding a more powerful model. We explain this further in the expressivity analysis (Section III-D). In implementation, we adopt the hybrid graph transformer GPS [16], replacing full attention with GSSC (Fig. 2).

C. Extensions and Discussions

Selection Mechanism: To improve the selection/content-aware ability of SSMs, [33] proposed to make coefficients $\bar{A}, \bar{B}$ in Eq.(1) data-dependent, i.e., replacing $\bar{A}$ by $\bar{A}_i := \bar{A}(x_i)$ and $\bar{B}$ by $\bar{B}_i := \bar{B}(x_i)$. This leads to a data-dependent convolution: $h_i = \sum_{j=1}^i \bar{A}_{i-1} \bar{A}_{i-2} \dots \bar{A}_j \bar{B}_j x_j$. Again, this convolution can be factorized into $h_i = \bar{A}_i (\sum_{j=1}^i \bar{A}_j^{-1} \bar{B}_j x_j)$, where $\bar{A}_i := \bar{A}_{i-1} \bar{A}_{i-2} \dots \bar{A}_1$ depends on features and positions of all preceding tokens of token i . We can generalize this “data-dependent position” idea to GSSC, defining a data-dependent positional encodings $\tilde{z}$ as follows:

$$\tilde{z}_u = \sum_{v \in \mathcal{V}} \langle z_u \mathbf{W}_{dq}, z_v \mathbf{W}_{dk} \rangle (z_v \odot x_v) \mathbf{W}_{dv}, \quad (6)$$

where $z_v \odot x_v$ means first broadcasting x_v from $\mathbb{R}^m$ to $\mathbb{R}^{d \times m}$ and then doing the element-wise product with z_v . The new PEs $\tilde{z}_u$ rely on the features and PEs of all nodes, which preserves permutation equivariance. If introducing this selection mechanism, we simply replace every z_u in Eq. (4) by $\tilde{z}_u$. Thanks to the factorizable kernel $\langle z_u \mathbf{W}_{dq}, z_v \mathbf{W}_{dk} \rangle$, computing $\tilde{z}_u$ can still be done in $\mathcal{O}(nmd^2)$, linear w.r.t. graph size.

Compared to Graph Spectral Convolution: GSSC may also look similar to graph spectral convolution, usually in the form $\mathbf{H} = \mathbf{V} \psi(\Lambda) \mathbf{V}^\top \mathbf{X} \mathbf{W}$, where $\mathbf{H} = [h_1^\top; \dots; h_n^\top] \in \mathbb{R}^{n \times m}$ and $\mathbf{X} = [x_1^\top; \dots; x_n^\top] \in \mathbb{R}^{n \times m}$ are row-wise concatenations of features, $\psi(\Lambda) = \text{diag}([\psi(\lambda_1), \dots, \psi(\lambda_n)])$ is spectrum-domain filtering with $\psi : \mathbb{R} \rightarrow \mathbb{R}$ an element-wise function. Equivalently, it can be written as $h_u = \sum_{v \in \mathcal{V}} \langle p_u, \psi(\Lambda) \odot p_v \rangle \mathbf{W} x_v$. There are several differences between GSSC (Eq. (4)) and spectral convolution: (1) $\psi : \mathbb{R} \rightarrow \mathbb{R}$ is

element-wise, while $\phi : \mathbb{R}^m \rightarrow \mathbb{R}^m$ is a general permutation equivariant function considering mutual interactions between frequencies; (2) GSSC distinguishes a node itself (weights $\mathbf{W}_{sq}, \mathbf{W}_{sk}, \mathbf{W}_{so}$) and other nodes (weights $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_o$), while spectral convolution treats all nodes with the same weight $\mathbf{W}$. The former helps express diagonal element extraction ($\text{diag}(\mathbf{A}^k)$) in cycle counting. By contrast, spectral convolution with non-distinguishable node features cannot be more powerful than 1-WL [57], while GSSC is more powerful as shown later.

Super-linear Computation of Laplacian PEs: GSSC requires the computation of Laplacian eigendecomposition as preprocessing. Although finding all eigenvectors and eigenvalues can be costly, (1) in real experiments, such preprocessing can be done efficiently, which may only occupy less than 10% wall-clock time of the entire training process (see Section V-C for quantitative results on real-world datasets); (2) we only need top- d eigenvectors and eigenvalues, which can be efficiently found by Lanczos methods [58] with complexity $O(Ed)$ (E is the number of edges) or similarly by LOBPCG methods [59]; (3) one can also adopt random Fourier feature-based approaches to fast approximate those kernels' factorization [60], [61].

D. Expressive Power

We measure GSSC's expressivity via its graph distinguishing ability and ability to count graph substructures.

Proposition III.2. There exist two non-isomorphic graphs, $\mathcal{G}$ and $\mathcal{G}'$, that are indistinguishable by the WL test but can be distinguished by GSSC (Eq. (4)). Furthermore, all non-isomorphic graphs distinguishable by GSSC can also be distinguished by the 3-WL test.

Proposition III.3 (Counting paths and cycles). GSSC (Eq. (4)) can count 3-paths and 3-cycles. With selection mechanism Eq.(6), it can at least count 4-paths and 4-cycles.

GSSC Enhances MPNNs: The results above show that combining MPNNs with GSSC yields a more powerful model.

MPNNs Help GSSC: GSSC excels at long-range interactions but may struggle to fully encode local information when using only a small number of eigenvectors. MPNNs address this by capturing local structure through neighborhood aggregation.

IV. RELATED WORKS

Efficient Graph Transformers: To reduce the quadratic complexity of graph attention, [16], [25], [27] adopt linear attention, e.g., factorizing the kernel into products of Random Positive Features [21], [29], [62] replace attention with inner products of learnable node features, which omit positional encodings but add message passing layers to encode topology. For comparison, GSSC (Eq.(4)), derived from SSMs, is based on factorized convolution kernels, in a similar spirit. Recent work has shown equivalences between linear transformers and SSMs [42]. However, GSSC is distinct from linear graph transformers: while the latter approximate a specific attention kernel (softmax with inner product) via random features, GSSC constructs convolution kernels in a learnable, stable manner, not limited to a specific kernel factorization. [26] use virtual global nodes and expander graphs for sparse attention; [28] applies a projection matrix to reduce the graph size factor n to k . Both are fundamentally different from GSSC.

State Space Models: Classic SSMs [31] describe the evolution of state variables using first-order differential equations, providing a unified framework for time series modeling. To capture long-range dependencies, Structural SSMs (S4) with a structural matrix $\bar{A}$ are introduced, e.g., HiPPO [34] and diagonal matrices [63]. Many variants [33], [64], [65] have also been proposed. Note that GSSC (Eq. (4)) does not rely on a structural matrix $\bar{A}$ for long-range dependencies. $\bar{A}$

describes the causal (recurrence) relation Eq. (1) for sequences, while no such causality exists for graphs. Instead, GSSC generalizes from the SSM convolution Eq. (2), with behavior determined by the design of its convolution kernel, i.e., the inner product of learnable graph PEs. Long-range properties can be achieved by choosing ϕ functions, as shown in Proposition III.1.

State Space Models for Graphs: There have been efforts to replace attention in graph transformers with SSMs, mainly by sequentializing graphs and applying models such as Mamba [33]. Graph-Mamba-I [36] sorts nodes by degree and applies Mamba, but degree multiplicity requires random permutations during training, breaking node-order equivariance. Graph-Mamba-II [37] builds sequences of K -hop subgraphs rooted at each node, encodes them with a GNN, and applies Mamba hierarchically, though the final Mamba layer also breaks permutation invariance. [66] treat K -hop neighbors as sequences using DeepSets [54], but incur high overhead since each subgraph token must be encoded individually. Other works include heterogeneous graph modeling via metapath-based tokenization [67], spectral convolutions using SSMs on ordered frequencies [68], and random-walk-based SSM encoders [69]. In contrast, GSSC avoids graph sequentialization, generalizing SSC to graphs while preserving permutation equivariance, long-range dependencies, and linear complexity.

Recent work on graph prompting [70], [71], [72] studies adaptation of pretrained graph neural networks and may also employ structural encodings. They address a different problem setting to the graph state space models.

V. EXPERIMENTS

We evaluate GSSC on 12 datasets against strong baselines, focusing on four key questions:

- Q1 How expressive in counting graph substructures?
- Q2 How effective in capturing long-range dependencies?
- Q3 How does GSSC perform on general graph benchmarks compared to other baselines?
- Q4 How does the computational time/space of GSSC scale with graph size?

Below we briefly introduce the datasets, model implementation, and baselines (detailed description in Appendix B in supplementary material, available online).

Datasets: To answer Q1, we use graph substructure counting datasets from [14], [73], [74]. It contains 5k graphs from different distributions (see [14] Appendix M.2.1, available online), and the task is to predict the number of cycles as node-level regression. To answer Q2, we evaluate GSSC on the Long Range Graph Benchmark [13], for which we adopt Peptides-func, Peptides-struct, and PascalVOC-SP. To answer Q3, we use molecular datasets (ZINC [39], ogbg-molhiv [38]), image datasets (MNIST, CIFAR10 [39]), synthetic datasets (PATTERN, CLUSTER [39]), and function call graphs (MalNet-Tiny [75]). To answer Q4, we evaluate the runtime on a random graph dataset with graph sizes from 1k to 60k. The full dataset statistics can be found at Appendix Table B.I, available online.

GSSC Implementation: As discussed in Section III-B, we implement a GSSC hybrid model by replacing transformer layers in GraphGPS [16] with GSSC layers. We adopt DeepSets [54] to model ϕ in Eq.(4). Selective mechanism Eq. (6) is introduced *exclusively* to cycle-counting tasks. The $d = 32$ smallest eigenvalues and their eigenvectors are adopted for all datasets except molecular ones, which employ $d = 16$.

Baselines: We consider various baselines that can be categorized into: (1) MPNNs: GCN [7], GIN [9], GAT [8], Gated GCN [76] and PNA [77]; (2) Subgraph GNNs: NGNN [48], ID-GNN [47], GIN-AK+ [73], I²-GNN [74]; (3) Graph transformers: Graphormer [19],

TABLE I
BENCHMARK ON GNN BENCHMARK & LONG RANGE GRAPH BENCHMARK. **Bold[†]**, **Bold[‡]**, AND **Bold** DENOTE THE FIRST, SECOND, AND THIRD BEST RESULTS, RESPECTIVELY. RESULTS ARE REPORTED AS MEAN_{±STD}.

	MNIST	CIFAR10	PATTERN	CLUSTER	MalNet-Tiny	PascalVOC-SP	Peptides-func	Peptides-struct
	Accuracy ↑	Accuracy ↑	Accuracy ↑	Accuracy ↑	Accuracy ↑	F1 score↑	AP ↑	MAE ↓
GCN	90.705±0.218	55.710±0.381	71.892±0.334	68.498±0.976	81.0	0.2078±0.0031	0.6860±0.0050	0.2460 ±0.0007
GIN	96.485±0.252	55.255±1.527	85.387±0.136	64.716±1.553	88.98±0.56	0.2718±0.0054	0.6621±0.0067	0.2473±0.0017
GAT	95.535±0.205	64.223±0.455	78.271±0.186	70.587±0.447	92.10±0.24	—	—	—
GatedGCN	97.340±0.143	67.312±0.311	85.568±0.088	73.840±0.326	92.23±0.65	0.3880±0.0040	0.6765±0.0047	0.2473±0.0017
SAN	—	—	86.581±0.037	76.691±0.650	—	0.3216±0.0027	0.6439±0.0075	0.2545±0.0012
GraphGPS	98.051±0.126	72.298±0.356	86.685±0.059	78.016±0.180	93.50 ±0.41	0.4440 ±0.0065	0.6535±0.0041	0.2500±0.0005
Exphormer	98.550 ±0.039	74.690±0.125	86.740±0.015	78.070±0.037	94.02 ±0.21	0.3975±0.0037	0.6527±0.0043	0.2481±0.0007
Grit	98.108±0.111	76.468±0.881	87.196 ±0.076	80.026 ±0.277	—	—	0.6988±0.0082	0.2460 ±0.0012
GRED	98.383±0.012	76.853 ±0.185	86.759±0.020	78.495 ±0.103	—	—	0.7133 ±0.0011	0.2455 ±0.0013
Graph-Mamba-I	98.420±0.080	73.700±0.340	86.710±0.050	76.800±0.360	93.40±0.27	0.4191±0.0126	0.6739±0.0087	0.2478±0.0016
NeuralWalker	98.692 ±0.079	76.903 ±0.457	86.977 ±0.012	78.189±0.188	—	0.4912 ±0.0042	0.7096 ±0.0078	0.2463±0.0005
GSSC	98.492 ±0.051	77.642 ±0.456	87.510 ±0.082	79.156 ±0.152	94.06 ±0.64	0.4561 ±0.0039	0.7101 ±0.0062	0.2459 ±0.0020

TABLE II
BENCHMARK ON GRAPH SUBSTRUCTURE COUNTING (NORMALIZED MAE ↓).
SELECTIVE MECHANISM IS APPLIED.

	3-Cycle	4-Cycle	5-Cycle
GIN	0.3515	0.2742	0.2088
ID-GIN	0.0006	0.0022	0.0490
NGNN	0.0003	0.0013	0.0402
GIN-AK+	0.0004	0.0041	0.0133
I ² -GNN	0.0003	0.0016	0.0028
Exphormer	0.0006	0.0468	0.0827
Graph-Mamba-I	0.0014	0.0113	0.0301
GraphGPS (Transformer)	0.0007	0.0125	0.0297
GraphGPS (Performer)	0.0011	0.0131	0.0301
GSSC	0.0002	0.0013	0.0113

SAN [50], GraphGPS [16], Grit [51], Specformer [78]; (4) efficient models: Exphormer [26], GRED [66], Graph-Mamba-I [36]; (5) Others: SUN [79], SignNet [53], SPE [55] and NeuralWalkers [69]. We also compared Graph-Mamba-II [37], and GSSC outperforms it on all included datasets. But we put the results in Appendix C, available online as we cannot reproduce them due to the lack of code. We exclude Polynormer [62] as it adopts node classification tasks on homophilic and heterophilic graphs, none of which overlap with our chosen datasets.

A. Graph Substructure Counting

Table II shows the normalized Mean Absolute Error results. We adopt GIN backbone for all baseline subgraph models [74]. In terms of predicting 3-cycles and 4-cycles, GSSC achieves the best results compared to subgraph GNNs and I²-GNNs (models that can provably count 3,4-cycles), validating Theorem III.3. For the prediction of 5-cycles, GSSC greatly outperforms the MPNN and ID-GNN (models that cannot predict 5-cycles) reducing normalized MAE by 94.6% and 71.9%, respectively. Besides, GSSC achieves constantly better performance than GNN-AK+, a subgraph GNN model that is strictly stronger than ID-GNN and NGNN [74]. These results demonstrate the strong function-fitting ability of GSSC.

B. Graph Learning Benchmarks

Tables I and III evaluate GSSC on multiple widely used graph learning benchmarks. GSSC achieves the best performance on 5 datasets out of 11, and ranks top 3 in the other 6.

Long Range Graph Benchmark [13]: In Table I, GSSC achieves highly competitive results close to the leading benchmarks on PascalVOC-SP, Peptides-func and Peptides-struct, and it constantly has

TABLE III
BENCHMARK ON MOLECULAR DATASETS. **Bold[†]**, **Bold[‡]**, AND **Bold** DENOTE THE FIRST, SECOND, AND THIRD BEST RESULTS.

	ZINC-12k	ZINC-Full	ogbg-molhiv
	MAE ↓	MAE ↓	AUROC ↑
GCN	0.367±0.011	0.113±0.002	75.99±1.19
GIN	0.526±0.051	0.088±0.002	77.07±1.49
GAT	0.384±0.007	0.111±0.002	—
PNA	0.188±0.004	—	79.05±1.32
NGNN	0.111±0.003	0.029±0.001	78.34±1.86
GIN-AK+	0.080±0.001	—	79.61 ±1.19
I ² -GNN	0.083±0.001	0.023±0.001	78.68±0.93
SUN	0.083±0.003	0.024±0.003	80.03 ±0.55
Graphormer	0.122±0.006	0.052±0.005	—
SAN	0.139±0.006	—	77.85±2.47
GraphGPS	0.070±0.004	—	78.80±1.01
GraphGPS (Performer)	0.072±0.002	—	77.79±1.25
Exphormer	0.111±0.007	—	78.79±1.31
Specformer	0.066 ±0.003	—	78.89±1.24
SPE	0.070±0.004	—	—
SignNet	0.084±0.006	0.024±0.003	—
Grit	0.059 ±0.002	0.023±0.001	—
GRED	0.077±0.002	—	—
Graph-Mamba-I	0.067 ±0.002	—	78.23±1.21
GSSC	0.064 ±0.002	0.019 ±0.001	80.35 ±1.42

better performance than Graph-Mamba-I. These results show GSSC's ability to capture long-range dependencies. NeuralWalker [69] also presents competitive performance. However, NeuralWalker suffers from overheads of extracting a great number of long random walks to achieve reasonable generalization due to its violation of permutation invariance (See more details in Appendix C, available online).

Molecular Graph Benchmark [38], [39]: Table III shows GSSC achieves the best results on ZINC-Full and ogbg-molhiv and is competitive to the SOTA model on ZINC-12k.

GNN Benchmark [39] & MalNet-Tiny [75]: Table I shows that GSSC achieves *state-of-the-art* performance on CIFAR10, PATTERN, and MalNet-Tiny, and ranks second-best on CLUSTER and third-best on MNIST, demonstrating its superior capability on general graph-structured data.

Ablation study: Comparisons with GraphGPS and MPNNs serve as ablations, where GSSC is replaced by a transformer layer or removed while keeping other modules unchanged. GSSC consistently outperforms both, confirming its effectiveness as an alternative to full attention. Additional results, including the effect of eigenvector number, are in Appendix C, available online.

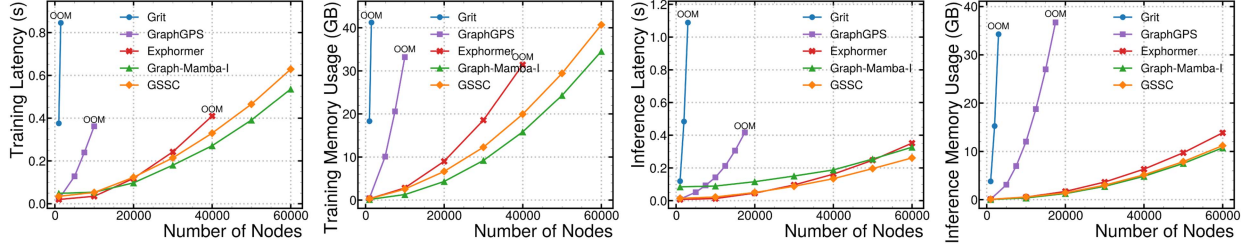


Fig. 3. Model training and inference costs per graph. Results are averaged over 100 runs.

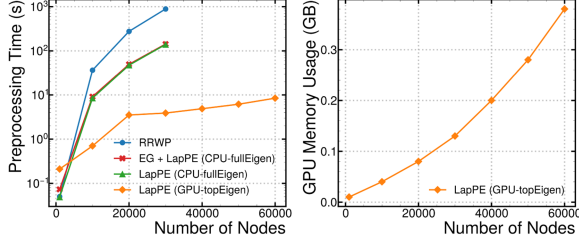


Fig. 4. Preprocessing costs per graph.

C. Computational Costs Comparison

The computational costs of graph learning methods consist of two main aspects: preprocessing (e.g., calculating PE), and model training/inference. To demonstrate the efficiency of GSSC, we benchmark its computational costs against 4 recent state-of-the-art methods, including GraphGPS [16], Grit [51], Exphormer [26], and Graph-Mamba-I [36]. Exphormer, Graph-Mamba-I, and GSSC are designed with linear complexity with respect to the number of nodes n , whereas Grit and GraphGPS exhibit quadratic complexity by design.

Benchmark Setup: We generate Erdős-Rényi random graphs with sizes from 1k to 60k. To simulate typical graph sparsity, we add $n^2 \times 1\%$ edges for graphs with fewer than 10k nodes and $n^2 \times 0.1\%$ edges for graphs with more than 10k nodes. On GPUs, we use `torch.utils.benchmark.Timer` and `torch.cuda.max_memory_allocated` to measure time and memory; on CPUs, we use `time.time`.

Preprocessing Costs: As all baselines preprocess on CPUs, we first examine CPU time to assess scalability (Fig. 4, left). Grit computes RRWP [51], which requires more time due to repeated multiplications of $n \times n$ matrices. Other methods, e.g., Exphormer, GraphGPS, Graph-Mamba-I, and GSSC, use Laplacian eigendecomposition for graph PE. For these, we follow the baselines and perform full eigendecomposition on CPUs (16 cores), shown by the green line. Exphormer additionally constructs expander graphs (EG), slightly increasing preprocessing time (red line). Clearly, full eigendecomposition is costly for large graphs. For large graphs, we can bypass it and leverage iterative GPU methods such as `torch.linalg.pcg` [59] to obtain top- d results, which is both fast and memory-efficient (linear in n), shown by the orange lines (LapPE GPU-topEigen) with $d = 32$. For small graphs, however, full eigendecomposition on CPUs remains efficient.

Training/Inference Costs: Fig. 3 benchmarks training and inference costs in inductive node classification. All methods use the same number of layers and 500k parameters. Methods marked “OOM” indicate out-of-memory errors on a 48 GB GPU when the node count increases by 5k. GSSC and Graph-Mamba-I are the most efficient. Although Graph-Mamba-I shows slightly better training efficiency, this comes from its

TABLE IV
COMPUTATIONAL COSTS OF GSSC ON REAL-WORLD DATASETS.

	ZINC-12k	PascalVOC-SP
Avg. # nodes	23.2	479.4
# graphs	12,000	11,355
# epochs	2,000	300
Training time per epoch	10.9 s	13.9 s
Total training time	6.1 h	1.2 h
Total preprocessing time	20.6 s	334.8 s
$\frac{\text{Total preprocessing time}}{\text{Total training time}}$	0.1%	7.6%

integration with the optimized Mamba API [33]. GSSC’s efficiency could also improve with hardware-aware optimization.

Computational Costs on Real-World Datasets: The benchmark evaluates graph transformers on large graphs for scalability, though in practice they are typically used on smaller graphs [16], where global dependencies matter. We report computational costs on two extremes: ZINC-12 k (smallest) and PascalVOC-SP (largest). Following prior work [16], [26], [51], preprocessing is done on CPUs per graph with identical training epochs. As Table IV shows, preprocessing is negligible for ZINC-12 k ($<1\%$ of >100 epochs) and remains efficient for PascalVOC-SP ($<10\%$ of training time). Eigendecomposition on GPUs could further reduce costs. Additional real-world million-scale results are in Appendix C, available online, where GSSC achieves lower inference latency with comparable accuracy.

VI. CONCLUSION

This work extends State Space Models (SSMs) to graphs by proposing GSSC, which uses permutation-equivariant aggregation and graph kernels based on positional encodings. GSSC inherits SSMs’ strengths: efficient computation, long-range dependency, and generalization across varying sizes.

ACKNOWLEDGMENT

The authors would like to thank Mufei Li, Haoyu Wang, Xiyuan Wang, and Peihao Wang. Mufei Li introduced the idea of SSM to the team. Xiyuan Wang shared a lot of insights in viewing graphs as point sets. The discussion with Peihao Wang raises the observations on how SSM essentially achieves its core three advantages and inspires this study.

REFERENCES

- [1] D. K. Duvenaud et al., “Convolutional networks on graphs for learning molecular fingerprints,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, vol. 28, pp. 2224–2232.

- [2] Y. Wang, Y. Min, E. Shao, and J. Wu, "Molecular graph contrastive learning with parameterized explainable augmentations," in *Proc. IEEE Int. Conf. Bioinf. Biomed.*, 2021, pp. 1558–1563.
- [3] J. Xiong, Z. Xiong, K. Chen, H. Jiang, and M. Zheng, "Graph neural networks for automated de Novo drug design," *Drug Discov. Today*, vol. 26, no. 6, pp. 1382–1393, 2021.
- [4] J. M. Stokes et al., "A deep learning approach to antibiotic discovery," *Cell*, vol. 180, no. 4, pp. 688–702, 2020.
- [5] W. Fan et al., "Graph neural networks for social recommendation," in *Proc. World Wide Web Conf.*, 2019, pp. 417–426.
- [6] Z. Guo and H. Wang, "A deep graph neural network-based mechanism for social recommendations," *IEEE Trans. Ind. Informat.*, vol. 17, no. 4, pp. 2776–2783, Apr. 2021.
- [7] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2016, pp. 2713–2726.
- [8] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 2920–2931.
- [9] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 9104–9120.
- [10] C. Morris et al., "Weisfeiler and Leman go neural: Higher-order graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, pp. 4602–4609.
- [11] J. Topping, F. D. Giovanni, B. P. Chamberlain, X. Dong, and M. M. Bronstein, "Understanding over-squashing and bottlenecks on graphs via curvature," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 8247–8276.
- [12] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, and X. Sun, "Measuring and relieving the over-smoothing problem for graph neural networks from the topological view," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, pp. 3438–3445.
- [13] V. P. Dwivedi et al., "Long range graph benchmark," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 22326–22340.
- [14] Z. Chen, L. Chen, S. Villar, and J. Bruna, "Can graph neural networks count substructures?," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 10383–10395.
- [15] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 5998–6008.
- [16] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a general, powerful, scalable graph transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 14501–14515.
- [17] P. Li, Y. Wang, H. Wang, and J. Leskovec, "Distance encoding: Design provably more powerful neural networks for graph representation learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 4465–4478.
- [18] H. Wang, H. Yin, M. Zhang, and P. Li, "Equivariant and stable positional encoding for more powerful graph neural networks," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 17029–17052.
- [19] C. Ying et al., "Do transformers really perform badly for graph representation?," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 28877–28888.
- [20] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNs: Fast autoregressive transformers with linear attention," in *Proc. 37th Int. Conf. Mach. Learn.*, H. D. III and A. Singh, Eds., Jul. 2020, pp. 5156–5165.
- [21] K. M. Choromanski et al., "Rethinking attention with performers," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1060–1097.
- [22] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proc. 30th Annu. ACM Symp. Theory Comput.*, 1998, pp. 604–613.
- [23] N. Kitaev, L. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 12764–12775.
- [24] S. Arora et al., "Simple linear attention language models balance the recall-throughput tradeoff," in *Proc. 41st Int. Conf. Mach. Learn.*, vol. 235, 2024, pp. 1763–1840.
- [25] Q. Wu, W. Zhao, Z. Li, D. P. Wipf, and J. Yan, "NodeFormer: A scalable graph structure learning transformer for node classification," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 27387–27401.
- [26] H. Shirzad, A. Vellingker, B. Venkatachalam, D. J. Sutherland, and A. K. Sinop, "Expformer: Sparse transformers for graphs," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 31613–31632.
- [27] Y. Wu et al., "KDLGT: A linear graph transformer framework via kernel decomposition approach," in *Proc. 32nd Int. Joint Conf. Artif. Intell.*, 2023, pp. 2370–2378.
- [28] K. Kong, J. Chen, J. Kirchenbauer, R. Ni, C. B. Bruss, and T. Goldstein, "GOAT: A global transformer on large-scale graphs," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 17375–17390.
- [29] Q. Wu et al., "Simplifying and empowering transformers for large-graph representations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 36, pp. 64753–64773.
- [30] S. Miao, Z. Lu, M. Liu, J. Duarte, and P. Li, "Locality-sensitive hashing-based efficient point transformer with applications in high-energy physics," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 35546–35569.
- [31] R. E. Kalman, "A new approach to linear filtering and prediction problems," *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, Mar. 1960.
- [32] A. Gu, K. Goel, and C. Re, "Efficiently modeling long sequences with structured state spaces," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 14323–14349.
- [33] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *Proc. 1st Conf. Lang. Model.*, 2025, pp. 173–184.
- [34] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré, "Hippo: Recurrent memory with optimal polynomial projections," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 1474–1487.
- [35] Y. Tay et al., "Long range arena: A benchmark for efficient transformers," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 11694–11712.
- [36] C. Wang, O. Tsepa, J. Ma, and B. Wang, "Graph-mamba: Towards long-range graph sequence modeling with selective state spaces," 2024, *arXiv:2402.00789*.
- [37] A. Behrouz and F. Hashemi, "Graph mamba: Towards learning on graphs with state space models," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 119–130.
- [38] W. Hu et al., "Open graph benchmark: Datasets for machine learning on graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 22118–22133.
- [39] V. P. Dwivedi, C. K. Joshi, A. T. Luu, T. Laurent, Y. Bengio, and X. Bresson, "Benchmarking graph neural networks," *J. Mach. Learn. Res.*, vol. 24, no. 43, pp. 1–48, 2023.
- [40] R. Pascanu, "On the difficulty of training recurrent neural networks," in *Proc. 30th Int. Conf. Mach. Learn.*, 2013, vol. 28, no. 3, pp. 1310–1318.
- [41] M. Karami and A. Ghodsi, "Orchid: Flexible and data-dependent convolution for sequence modeling," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 37, pp. 76991–77022.
- [42] T. Dao and A. Gu, "Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 10041–10071.
- [43] C. L. Giles and T. Maxwell, "Learning, invariance, and generalization in high-order neural networks," *Appl. Opt.*, vol. 26, no. 23, pp. 4972–4978, 1987.
- [44] L. Page, S. Brin, R. Motwani, and T. Winograd, "The pagerank citation ranking: Bringing order to the web," in *Proc. 7th Int. World Wide Web Conf.*, 1998, pp. 161–172.
- [45] F. Chung, "The heat kernel as the pagerank of a graph," *Proc. Nat. Acad. Sci.*, vol. 104, no. 50, pp. 19735–19740, 2007.
- [46] J. L. Palacios, "Resistance distance in graphs and random walks," *Int. J. Quantum Chem.*, vol. 81, no. 1, pp. 29–33, 2001.
- [47] J. You, J. M. Gomes-Selman, R. Ying, and J. Leskovec, "Identity-aware graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, pp. 10737–10745.
- [48] M. Zhang and P. Li, "Nested graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 15734–15747.
- [49] E. Chien, J. Peng, P. Li, and O. Milenkovic, "Adaptive universal generalized pagerank graph neural network," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 16138–16161.
- [50] D. Kreuzer, D. Beaini, W. Hamilton, P. Létourneau, and P. Tossou, "Rethinking graph transformers with spectral attention," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 21618–21629.
- [51] L. Ma et al., "Graph inductive biases in transformers without message passing," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 23321–23337.
- [52] M. Belkin and P. Niyogi, "Laplacian eigenmaps for dimensionality reduction and data representation," *Neural Computation*, vol. 15, no. 6, pp. 1373–1396, Jun. 2003.
- [53] D. Lim et al., "Sign and basis invariant networks for spectral graph representation learning," in *Proc. 11th Int. Conf. Learn. Representations*, 2022, pp. 10683–10726.

- [54] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. R. Salakhutdinov, and A. J. Smola, "Deep sets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 3391–3401.
- [55] Y. Huang et al., "On the stability of expressive positional encodings for graph neural networks," in *Proc. 12th Int. Conf. Learn. Representations*, 2024, pp. 45657–45686.
- [56] D. Chen, L. O'Bray, and K. Borgwardt, "Structure-aware transformer for graph representation learning," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 3469–3489.
- [57] X. Wang and M. Zhang, "How powerful are spectral graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 23341–23362.
- [58] C. Lanczos, "An iteration method for the solution of the eigenvalue problem of linear differential and integral operators," *J. Res. Nat. Bur. Standards*, vol. 45, no. 4, pp. 255–282, 1950.
- [59] A. V. Knyazev, "Toward the optimal preconditioned eigensolver: Locally optimal block preconditioned conjugate gradient method," *SIAM J. Sci. Comput.*, vol. 23, no. 2, pp. 517–541, 2001.
- [60] A. J. Smola and R. Kondor, "Kernels and regularization on graphs," in *Proc. Learn. Theory Kernel Machines*, 2003, pp. 144–158.
- [61] K. M. Choromanski, "Taming graph kernels with random features," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 5964–5977.
- [62] C. Deng, Z. Yue, and Z. Zhang, "Polynormer: Polynomial-expressive graph transformer in linear time," in *Proc. 12th Int. Conf. Learn. Representations*, 2024, pp. 18954–18979.
- [63] A. Gupta, A. Gu, and J. Berant, "Diagonal state spaces are as effective as structured state spaces," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 22982–22994.
- [64] H. Mehta, A. Gupta, A. Cutkosky, and B. Neyshabur, "Long range language modeling via gated state spaces," in *Proc. 11th Int. Conf. Learn. Representations*, 2022, pp. 29000–29019.
- [65] D. Y. Fu, T. Dao, K. K. Saab, A. W. Thomas, A. Rudra, and C. Re, "Hungry hungry hippos: Towards language modeling with state space models," in *Proc. 11th Int. Conf. Learn. Representations*, 2022, pp. 12193–12217.
- [66] Y. Ding, A. Orvieto, B. He, and T. Hofmann, "Recurrent distance filtering for graph representation learning," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024.
- [67] Z. Pan, Y. Jeong, X. Liu, and H. Liu, "HeteGraph-mamba: Heterogeneous graph learning via selective state space model," 2024, *arXiv:2405.13915*.
- [68] G. Zhao, T. Wang, Y. Jin, C. Lang, Y. Li, and H. Ling, "GrassNet: State space model meets graph neural network," *Pattern Recognit.*, vol. 176, 2026, Art. no. 113197.
- [69] D. Chen, T. H. Schulz, and K. Borgwardt, "Learning long range dependencies on graphs via random walks," in *Proc. 13th Int. Conf. Learn. Representations*, 2025, pp. 51635–51661.
- [70] X. Sun, H. Cheng, J. Li, B. Liu, and J. Guan, "All in one: Multi-task prompting for graph neural networks," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 2120–2131.
- [71] C. Zi, H. Zhao, X. Sun, Y. Lin, H. Cheng, and J. Li, "ProG: A graph prompt learning benchmark," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 37, pp. 95406–95437.
- [72] Q. Wang, X. Sun, and H. Cheng, "Does graph prompt work? A data operation perspective with theoretical analysis," in *Proc. 42nd Int. Conf. Mach. Learn.*, 2025, pp. 64377–64402.
- [73] L. Zhao, W. Jin, L. Akoglu, and N. Shah, "From stars to subgraphs: Uplifting any GNN with local structure awareness," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 23733–23754.
- [74] Y. Huang, X. Peng, J. Ma, and M. Zhang, "Boosting the cycle counting power of graph neural networks with i^2 -GNNs," in *Proc. 11th Int. Conf. Learn. Representations*, 2022, pp. 25446–25472.
- [75] S. Freitas, Y. Dong, J. Neil, and D. H. Chau, "A large-scale database for graph representation learning," in *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks*, vol. 1, 2021, pp. 40–52.
- [76] X. Bresson and T. Laurent, "Residual gated graph convNets," 2017, *arXiv:1711.07553*.
- [77] G. Corso, L. Cavalleri, D. Beaini, P. Liò, and P. Veličković, "Principal neighbourhood aggregation for graph nets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 13260–13271.
- [78] D. Bo, C. Shi, L. Wang, and R. Liao, "Specformer: Spectral graph neural networks meet transformers," in *Proc. 11th Int. Conf. Learn. Representations*, 2022, pp. 35433–35450.
- [79] F. Frasca, B. Bevilacqua, M. Bronstein, and H. Maron, "Understanding and extending subgraph GNNs by rethinking their symmetries," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 31376–31390.